

The *AI-native* delivery system

A governed lifecycle from client intent to production evidence. Designed for delivery teams that use agents at machine speed without surrendering judgment, accountability, or standards.

EXACT PROCESS MAP

The graph in this document is the operating contract: each state has an owner, an entry trigger, an agent action, an evidence requirement, and an explicit authority to advance or refuse.

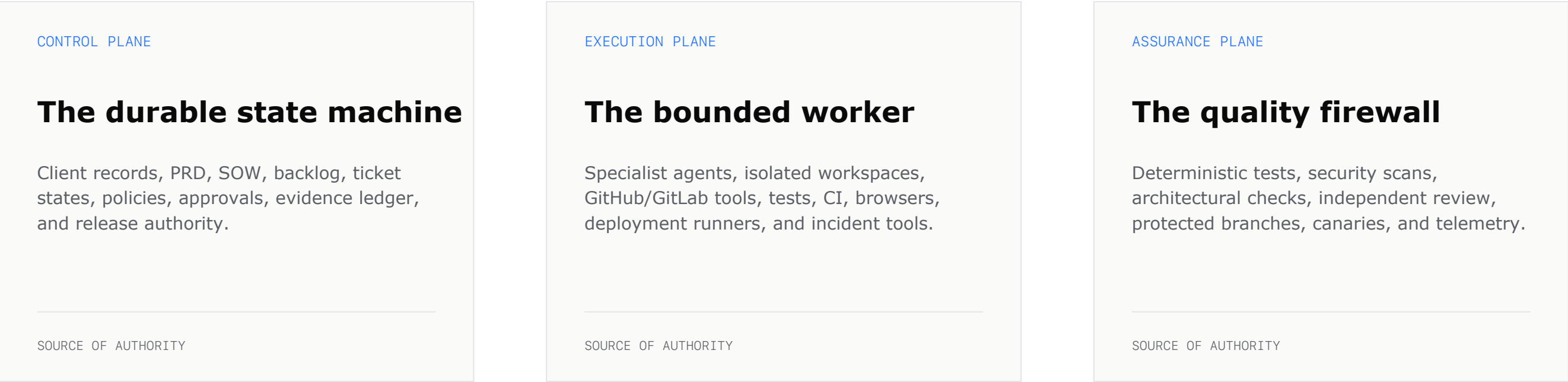
THE OPERATING THESIS

Humans set direction. Agents create and test changes. Controllers verify evidence. Protected systems decide what can move forward.

DOCTRINE v1.0 · APRIL 2026

A system of contracts, not prompts

The lifecycle is governed by three planes. The agent is never the source of truth; it is a worker operating inside a versioned control system.



WHAT IS AUTOMATED

- Discovery synthesis and ambiguity extraction**
Bounded by the current state, repository policy, and explicit acceptance criteria.
- PRD, story, task, and test-plan drafting**
Bounded by the current state, repository policy, and explicit acceptance criteria.
- Repository navigation and implementation**
Bounded by the current state, repository policy, and explicit acceptance criteria.
- Test generation, CI repair, documentation**
Bounded by the current state, repository policy, and explicit acceptance criteria.
- Review preparation and standards-drift detection**
Bounded by the current state, repository policy, and explicit acceptance criteria.

WHAT REMAINS HUMAN

- Problem selection and client truth**
Accountability cannot be delegated to model completion prose.
- SOW scope, price, dates, and legal terms**
Accountability cannot be delegated to model completion prose.
- Architecture and high-blast-radius decisions**
Accountability cannot be delegated to model completion prose.
- Security, privacy, tenancy, money, migrations**
Accountability cannot be delegated to model completion prose.
- Final release, rollback, and exceptions**
Accountability cannot be delegated to model completion prose.

THE CTO RULE

Let agents propose evidence and changes. Let deterministic systems verify them. Let humans approve irreversible boundaries.

The AI-native SDLC: one control-flow graph

The master LLD. Read left to right: each column names the state, trigger hook, responsible agent or human, concrete tool surface, output artifact, and gate that permits the next state.

01 / INTAKE	02 / DISCOVER	03 / CONTRACT	04 / DESIGN	05 / ASSIGN	06 / BUILD	07 / ASSURE	08 / OPERATE
HUMAN AUTHORITY / STATE							
Client sponsor	PO + client	PO + CTO	architect	delivery lead	delivery lead	reviewers	release owner
problem + constraints	truth / unknowns	outcome / price / date	risk / architecture	owner / capacity	ambiguity only	risk acceptance	exposure / rollback
HUMAN DECISION	HUMAN DECISION	HUMAN DECISION	HUMAN DECISION	HUMAN DECISION	HUMAN DECISION	HUMAN DECISION	HUMAN DECISION
AGENT / CONTROLLER / TOOL							
intake agent	research + product agent	PRD-SOW drafter	planner + threat agent	lifecycle controller	coding agent	test + review agents	deploy + SRE agents
CRM / transcript	Claude / Gemini / MCP	Rovo / AI-DLC	ADR / CodeQL rules	Linear / Jira / GitHub App	Codex / Claude / Copilot / Jules	Actions / CI / SAST / CODEOWNERS	protected branch / canary / SLO
WORKER / ROUTER	WORKER / ROUTER	WORKER / ROUTER	WORKER / ROUTER	WORKER / ROUTER	WORKER / ROUTER	WORKER / ROUTER	WORKER / ROUTER
HOOK / ARTIFACT / EXIT GATE							
discovery packet	locked discovery	versioned PRD + SOW	DAG + ADR + NFR	ticket + identity	branch + commits	evidence bundle	receipt + learning
issue.created	approve / clarify	signed scope	architecture gate	assign / @mention	push / PR.opened	checks + review	merge / alert / close
EXIT / NEXT TRIGGER	EXIT / NEXT TRIGGER	EXIT / NEXT TRIGGER	EXIT / NEXT TRIGGER	EXIT / NEXT TRIGGER	EXIT / NEXT TRIGGER	EXIT / NEXT TRIGGER	EXIT / NEXT TRIGGER
GLOBAL CONTROL RAIL							
repo policy: AGENTS.md / CLAUDE.md / WORKFLOW.md		runtime: sandbox + scoped identity + MCP/A2A			merge authority: protected branch		

LOOPS ambiguity -> human clarify | CI/review finding -> agent repair | deploy/SLO failure -> rollback or incident ticket | repeated failure -> rule, test, skill, or eval

Every state has a contract

The state machine is only useful if each transition is testable. The table below is the minimum contract for a client delivery lane.

STATE	ENTRY TRIGGER	AGENT WORK	HUMAN AUTHORITY	EXIT EVIDENCE
DRAFT	Client input / lead	Extract facts, unknowns, risks	Choose problem and sponsor	Discovery packet
DISCOVERY LOCKED	Facts reviewed	Normalize goals and constraints	Confirm client truth	Signed discovery record
PRD REFINED	Discovery complete	Draft requirements, AC, NFRs	Approve outcome and scope	Versioned PRD
SOW APPROVED	PRD accepted	Draft deliverables and assumptions	Approve price, dates, exclusions	Signed SOW
ARCHITECTURE APPROVED	SOW accepted	Propose architecture, ADR, threat model	Approve risk and design	Approved architecture packet
BACKLOG READY	Architecture accepted	Explode work into DAG	Prioritize and size	Linked epics / tasks
ASSIGNED	Ticket unblocked	Route by repo, domain, risk	Approve team/capacity exceptions	Owner + agent identity
IMPLEMENTING	Assignment claimed	Plan, edit, test, commit	Steer ambiguity	Branch + session log
VERIFYING	Commit pushed	Run declared checks and repair	Accept evidence for risk tier	Checks with checked/total
PR READY	Checks green	Summarize diff and traceability	Select reviewers	PR + evidence bundle
HUMAN APPROVED	Review complete	Address comments	Approve exact commit	Required approvals
CANARY	Merge / deploy	Observe health and rollback signals	Authorize exposure	Canary telemetry
RELEASED	Canary passes	Publish notes and update records	Release owner confirms	Deployment receipt
OBSERVING	Production live	Triage signals and drift	Decide rollback / follow-up	SLO and incident evidence
CLOSED	Outcome stable	Update standards and evals	Confirm acceptance and learnings	Closure ledger

CONTROL NOTE

A model saying 'done' never advances a state. The controller advances only when the evidence contract is present and valid.

The agent topology

Specialists are modular. The controller is singular. Human roles remain outside the worker pool and own the decision boundaries.

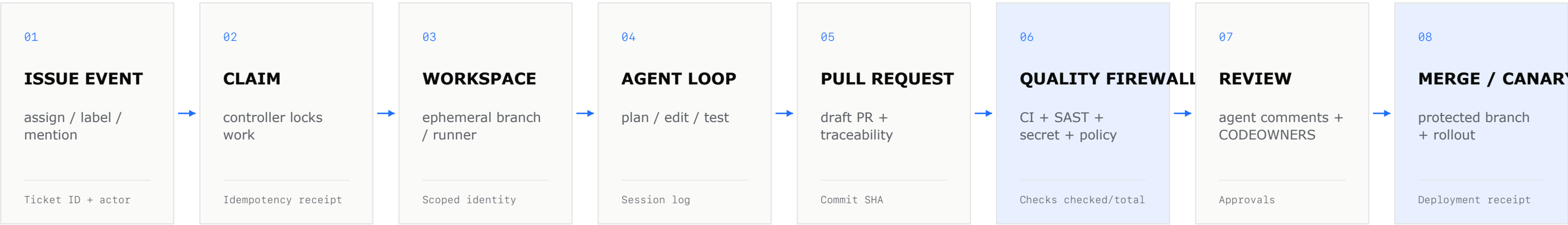


HUMAN ROLES / AUTHORITY

CLIENT SPONSOR	PRODUCT OWNER	SOLUTION ARCHITECT	DELIVERY LEAD	SECURITY / SRE	RELEASE OWNER
----------------	---------------	--------------------	---------------	----------------	---------------

From ticket to protected merge

The integration is an event-driven chain. Every external action is scoped, attributable, replayable, and reversible.



IDENTITY AND EVENT CONTRACT

INPUT

Issue content is untrusted data; the controller separates orchestration from untrusted context.

HOOKS

Use source-control events for lifecycle triggers and runtime hooks for tests, lint, policy, and secret checks.

CREDENTIALS

Use GitHub App or proxy-scoped credentials; never expose personal tokens or production secrets to the worker.

REVIEW

AI review is advisory; required approvals and protected-branch rules remain the merge authority.

FAILURE MODEL

Failure -> typed receipt -> retry or refusal -> human escalation. A zero-input check is a failure, not a green result.

Standards become executable

The design doctrine applies to the operating system too: one accent, explicit hierarchy, sharp boundaries, and no hidden authority.

QUALITY FIREWALL

Repository	AGENTS.md / CLAUDE.md / WORKFLOW.md	Versioned working contract
Architecture	ADR, dependency rules, structural tests	No silent drift
Code	Lint, type, schema, contract, migration checks	Deterministic correctness
Security	SAST, SCA, secrets, IaC, threat model	Risk before merge
Review	Independent agent + CODEOWNERS + human	Separation of duties
Runtime	Sandbox, egress allowlist, scoped identity	Bounded execution
Learning	Incident -> test / rule / skill / eval	Failure compounds into control

EVIDENCE RECORD

```
work_id      client-042 / feature-019
state        VERIFYING
actor        agent:coder-v3
commit       sha256:...
checks       checked=42 / total=42
approvals    security + codeowner
decision     controller: advance
```

RISK-TIERED AUTONOMY

T0 / LOW Docs, formatting, isolated tests Auto-run; deterministic checks; sampled human review	T1 / STANDARD Product code, ordinary refactors Agent PR; CI; independent review; human merge	T2 / HIGH Auth, tenancy, data, dependencies Architecture + security + human approval	T3 / IRREVERSIBLE Money, migrations, production, policy Explicit human authorization at exact commit/evidence
---	---	---	--

CTO operating rules

A compact policy for adopting this model without confusing autonomy with authority.

01

Make the ticket the durable unit

Sessions disappear. Work state, evidence, and ownership must survive the agent.

03

Keep SOW and PRD separate

The PRD describes product truth. The SOW describes contractual truth. They must link, not blur.

05

Treat external content as untrusted

Tickets, docs, webpages, and MCP results are inputs, not instructions with authority.

07

Review the exact commit

Approval attaches to a commit SHA and its evidence bundle, not to a moving branch.

09

Escalate irreversible actions

Production, security, money, tenancy, migrations, and policy require named human authority.

02

Make requirements testable

Every accepted requirement maps to a check, a reviewer, or an explicit human decision.

04

Use specialists plus one controller

Separate discovery, planning, coding, assurance, and operations workers; keep state authority singular.

06

Never let prose advance a gate

Only validated evidence can advance state; absent evidence is absent, never zero.

08

Let agents repair cheap failures

CI failures and review comments should loop back to the worker automatically.

10

Turn failure into a rule

Every repeated failure becomes a test, lint rule, skill, runbook, or evaluation.

DECISION

Adopt the model where the cost of context switching is high and the cost of verification is low. Keep humans close to ambiguity, risk, and irreversible change.

THE DEVX DOCTRINE · CTO EDITION

Research register: what to assemble

The graph is a CTO synthesis. The implementation primitives below are documented platform capabilities or company-reported practices; validate entitlements, security posture, and current product limits before rollout.

CONTROL PLANE	EXECUTION SURFACE	PROTOCOLS + FRAMEWORKS
GitHub Copilot agents Issues -> sessions -> PRs; agent review is advisory; branch protection and CODEOWNERS remain authoritative. GitLab Duo Agent Platform Planner, Developer, Code Review, CI/CD and custom flows; Mention/Assign triggers; audit events. Atlassian Rovo Natural language -> work items; agents can be assigned, mentioned, and connected through A2A. Linear + Symphony Issue status is the scheduler; isolated workspaces, DAG dependencies, retries, and WORKFLOW.md.	OpenAI Codex / App Server Common harness protocol for CLI, IDE and web; tools, approvals, threads, turns, and workspace isolation. Claude Code / Agent SDK CLAUDE.md, hooks, subagents, MCP, checkpoints, sandbox and scoped Git proxy. AWS Kiro / AI-DLC Adaptive Inception, Construction and Operations; specs and steering files; approval at phase boundaries. Google Jules / Gemini Async cloud VM, plan before edit, GitHub issue integration, PR delivery, scheduled and failure-repair tasks.	MCP Standard tool/context boundary; keep servers allowlisted, credentials scoped, and external content untrusted. A2A Agent-to-agent discovery and task handoff; use authenticated agent cards and explicit task ownership. SDK choices OpenAI Agents SDK, Google ADK, LangGraph, CrewAI, AutoGen/AG2, PydanticAI, OpenHands/SWE-agent. Repository contracts AGENTS.md, CLAUDE.md, WORKFLOW.md, Kiro steering, CODEOWNERS, required checks, protected branches.

METHODOLOGIES OBSERVED				
AI-DLC	Harness engineering	Spec-driven	TDD + repair loops	Evidence-bound Company OS
adaptive phases, approval checkpoints, audit trail	repo as system of record, executable invariants, agent-first work	PRD/spec -> tasks -> implementation -> verification	tests first, hooks/CI feedback, bounded autonomous repair	state machine, risk tiers, receipts, learning controls

PRIMARY SOURCES

openai.com/index/harness-engineering | openai.com/index/open-source-codex-orchestration-symphony | docs.github.com/copilot/agents | docs.gitlab.com/user/duo_agent_platform | aws.amazon.com/blogs/devops/open-sourcing-adaptive-workflows-for-ai-driven-development-life-cycle-ai-dlc | claude.com/blog/how-anthropic-teams-use-claude-code | blog.google/innovation-and-ai/models-and-research/google-labs/jules | modelcontextprotocol.io | a2a-protocol.org